

Отчет по лабораторной работе №1
по дисциплине «Вычислительная математика»

Студент гр. 5130904/30002

Кладковой М.Д.

Преподаватель
кандидат физико-математических наук

Воскобойников С.П.

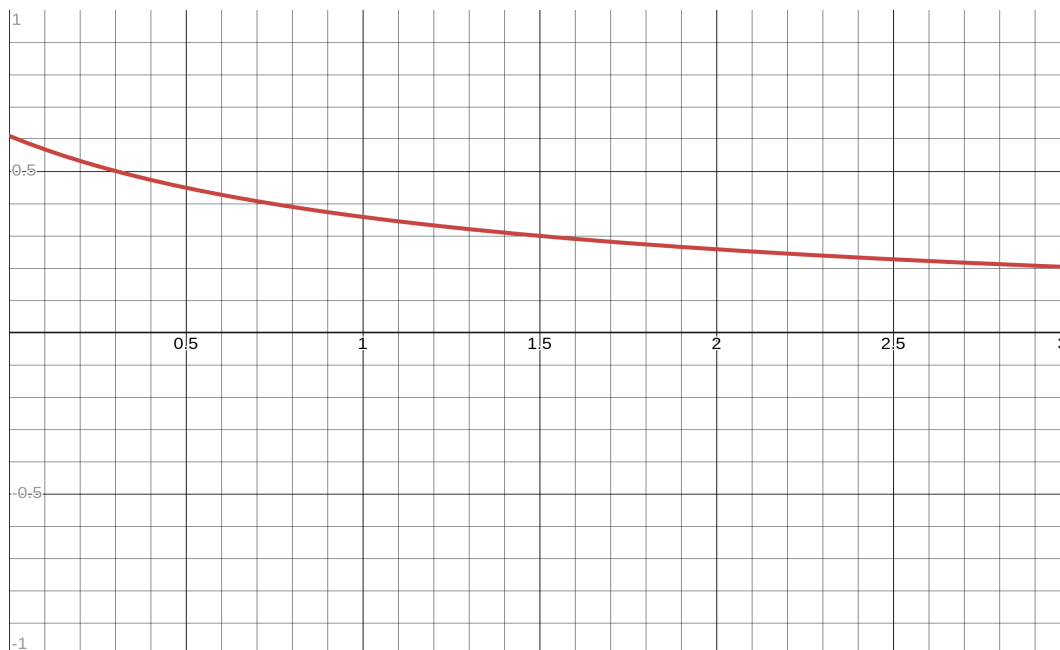
Условие:

ВАРИАНТ N 19

Для $0 \leq x \leq 3$ с $h = 0.5$ вычислить значения $f(x) = \int_0^1 \frac{dz}{e^{1.9z^2} + x}$, используя для

вычисления интеграла программу QUANC8. По полученным точкам построить сплайн-функцию и полином Лагранжа 6-й степени. Сравнить значения обеих аппроксимаций в точках $x_k = 0.25 + 0.5k$ ($k=0,1,\dots,5$).

График:



Текст программы:

```
#include <fstream>
#include <iomanip>
#include <iostream>
#include "forsythe.h"

double point;
double h = 0.5;
double x_beg = 0;
double x_end = 3;
int count_points = ((x_end - x_beg)/h) + 1;
double x_beg_compare = 0.25;

double func(double z) {
    return 1.0 / (exp(1.9*z*z) + point);
}

void computeQuanc8(double* x, double* f, int count_points, double h, double x_beg) {
    std::ofstream in("in.txt");
    double ans1, errest, flag;
    int col;

    std::cout << "Integral (quanc8)" << std::endl;
    for (int i = 0; i < count_points; i++) {
        point = x[i];
        QUANC8(func, 0.0, 1, 1e-7, 1e-7, ans1, errest, col, flag);

        std::cout << std::setw(3) << x[i] << std::setw(6) << "int=" << std::setw(8) << ans1
                  << std::setw(9) << "errest=" << errest << std::setw(6)
                  << "col=" << col << std::setw(7) << "flag=" << flag << std::endl;
        in << ans1 << std::endl;

        f[i] = ans1;
        if (i < (count_points - 1)) {
            x[i + 1] = x[i] + h;
        }
    }
}
```

```

    }
    in.close();
}

void compareQuanc8WithSpline(double* x, double* f, int count_points, double x_beg_compare) {
    std::ofstream s("s.txt"), q("q.txt");
    double ans1, ans2, errest, flag;
    int col;

    SPLINE spline(count_points, x, f);

    std::cout << "\nComparison of quanc8 and spline\n"
                << "-----\n" <<
                std::setw(20) << "quanc8" << std::setw(17) << "Spline" << std::setw(12) << "Q-S\n"
                << "-----" <<

std::endl;

    double xj = x_beg_compare;
    for (int j = 0; j < 5; j++) {
        point = xj;
        QUANC8(func, 0.0, 1, 1e-7, 1e-7, ans1, errest, col, flag);
        ans2 = spline.Eval(xj);
        std::cout << std::setw(8) << xj << " | "
                    << std::setw(12) << ans1 << " | "
                    << std::setw(12) << ans2 << " | "
                    << std::setw(12) << (ans1 - ans2) << std::endl;
        s << ans2 << std::endl;
        q << ans1 << std::endl;
        xj += 0.5;
    }
    s.close();
    q.close();
}

double omega (double x, int k) {
    double ans = 1;
    double xk = x_beg;
    for (int i = 0; i < count_points; i++) {
        if (i != k) {
            ans *= x - xk;
        }
        xk += h;
    }
    return ans;
}

double Lagrange (double x, double* f) {
    double ans = 0;
    double xk = x_beg;
    for (int i = 0; i < count_points; i++) {
        ans += (omega(x, i)/omega(xk, i))* f[i];
        xk += h;
    }
    return ans;
}

void compareQuanc8WithLagrange(double* x, double* f, double x_beg_compare) {
    std::ofstream l("l.txt");
    double ans1, ans2, errest, flag;
    int col;

    std::cout << "\nComparison of quanc8 and lagrange\n"
                << "-----\n" <<
                std::setw(20) << "quanc8" << std::setw(17) << "Lagrange" << std::setw(12) << "Q-L\n"
                << "-----" <<

std::endl;

    double xj = x_beg_compare;
    for (int j = 0; j < 5; j++) {
        point = xj;
        QUANC8(func, 0.0, 1.0, 1e-7, 1e-7, ans1, errest, col, flag);
        ans2 = Lagrange(xj, f);
        std::cout << std::setw(8) << xj << " | "
                    << std::setw(12) << ans1 << " | "
                    << std::setw(12) << ans2 << " | "
                    << std::setw(12) << (ans1 - ans2) << std::endl;
        l << ans2 << std::endl;
        xj += 0.5;
    }
}

```

```

    l.close();
}

int main() {
    double x[count_points], f[count_points];
    x[0] = x_beg;

    computeQuanc8(x, f, count_points, h, x_beg);

    compareQuanc8WithSpline(x, f, count_points, x_beg_compare);

    compareQuanc8WithLagrange(x, f, x_beg_compare);

    return 0;
}

```

Результат работы:

```

Integral (quanc8)
0   int=0.609984  errest=4.18898e-15  col=33  flag=0
0.5 int=0.448908  errest=2.59734e-13  col=33  flag=0
1   int=0.358368  errest=7.59304e-13  col=33  flag=0
1.5 int= 0.29956  errest=9.06762e-13  col=33  flag=0
2   int=0.257986  errest=5.00936e-13  col=33  flag=0
2.5 int=0.226901  errest=1.03904e-13  col=33  flag=0
3   int=0.202711  errest=4.63405e-13  col=33  flag=0

```

Comparison of quanc8 and spline

	quanc8	Spline	Q-S
0.25	0.516067	0.518045	-0.00197806
0.75	0.398189	0.397564	0.000625199
1.25	0.326178	0.32631	-0.000132227
1.75	0.277145	0.277099	4.59652e-05
2.25	0.241405	0.241397	7.92543e-06
2.75	0.2141	0.214149	-4.93956e-05

Comparison of quanc8 and lagrange

	quanc8	Lagrange	Q-L
0.25	0.516067	0.516557	-0.000489831
0.75	0.398189	0.398092	9.76743e-05
1.25	0.326178	0.326221	-4.28027e-05
1.75	0.277145	0.27711	3.53526e-05
2.25	0.241405	0.241459	-5.42117e-05
2.75	0.2141	0.213927	0.000173158

Вывод:

Были проведены вычисления функции на промежутке $[0;3]$ с использованием программы QUANC8. На основе полученных данных построены сплайн-функция и полином Лагранжа 6-й степени. Результаты вычисления функции, полученные с помощью полинома Лагранжа являются более точными, так как сплайн имеет большее отклонение от значений функции вычисляемых в точках на промежутке $[0.25; 1.75]$ с шагом 0.5. Но в точках $[2.25; 2.75]$ с шагом 0.5 сплайн оказался более точным.